

Dilema dvojitého agenta

- **Časový limit:** 12 minut.
- **Úložiště:** 5 GB
- **Prostředí:** jeden GPU (≈16 GB VRAM), bez internetu
- **Velikost řešení:** `solution.ipynb` ≤ 1 MB
- **Výchozí skóre:** 0

V národním centru umělé inteligence v Astaně analyzují fotografie dva počítačové modely — Model R (ResNet-18) a Model V (ViT-Tiny). V tuto chvíli oba modely pracují bezchybně, dosahují přesnosti 100% a shodují se u každého jednotlivého obrázku. Aby vedoucí vědec otestoval, nakolik se jejich chytré „mozky“ skutečně liší, zadá vám výzvu: proveďte v každé fotografii nepatrné, téměř neviditelné změny pixelů tak, aby se Model R a Model V vůbec neshodovaly.



1. Úloha

Dva předtrénované klasifikátory obrázků zpracovávají stejný obrázek. Na obrázcích poskytnutých v této úloze dosahují oba klasifikátory přesnosti 100%.

- **Model R:** `torchvision.models.resnet18` (CNN, ResNet18).
- **Model V:** `vit_tiny_patch16_224` od `timm` (Transformer, ViT-Tiny).

Vaším úkolem je pro každý obrázek vytvořit malou změnu („perturbaci“) tak, aby se oba modely neshodovaly. Pro každý obrázek musíte vytvořit **dvě různé** perturbace:

- **Typ A:** po jejím přidání Model R stále klasifikuje obrázek správně, ale Model V jej klasifikuje nesprávně.
- **Typ B:** po jejím přidání Model V stále klasifikuje obrázek správně, ale Model R jej klasifikuje nesprávně.

Každá perturbace musí být natolik *malá*, aby ji bylo obtížné postřehnout. Menší perturbace získávají vyšší skóre (viz oddíl 5). Perturbace se aplikuje přímo na původní obrázek na úrovni pixelů.

2. Veřejná data

S úlohou je poskytnuta sada obrázků rozdělená do dvou částí — `train` (100 obrázků) a `test_public` (100 obrázků) — přičemž obrázky mají různá rozlišení. Všechny obrázky pocházejí z 1000 tříd ImageNet-1K a Model R i Model V dosahují v obou částech přesnosti 100%.

Jsou poskytnuty následující soubory:

```
train/images/*.png      # 100 images in PNG format
train/labels.json       # maps each image index to its correct class
test_public/images/*.png # 100 images in PNG format
test_public/labels.json  # maps each image index to its correct class
```

Během vyhodnocování bude vaše složka `dataset/test_public/` transparentně nahrazena dvěma skrytými sadami obrázků (`test_leaderboard_a` a `test_leaderboard_b`) pro oficiální bodování. Každá z nich obsahuje **100 obrázků** ve formátu PNG a soubor s anotacemi.

Poznámka: Pro tuto úlohu jsou anotace v testovacích datasetech přístupné.

3. Výstupní formát

Pro každý obrázek musíte vytvořit dva soubory:

```
{index}_a.pt  # Type A perturbation
{index}_b.pt  # Type B perturbation
```

- `{index}` (0, 1, 2, ...), odpovídá názvu obrázku v datasetech.
- Každý soubor obsahuje jediný tenzor uložený pomocí `torch.save`. Jeho tvar musí být $3 \times H \times W$, kde H a W odpovídají **původnímu** rozlišení daného obrázku (nikoli 224×224).
- Kód by měl vytvořit pouze jeden soubor ZIP, `submission.zip`. Všechny soubory `.pt` umístěte do kořenové úrovně archivu ZIP bez nadřazené složky nebo podadresářů.

```
submission.zip
├─ 0_a.pt
├─ 0_b.pt
├─ 1_a.pt
└─ ...
```

Notebook vás upozorní, pokud se ve výstupním formátu vyskytnou jakékoli problémy.

4. Omezení

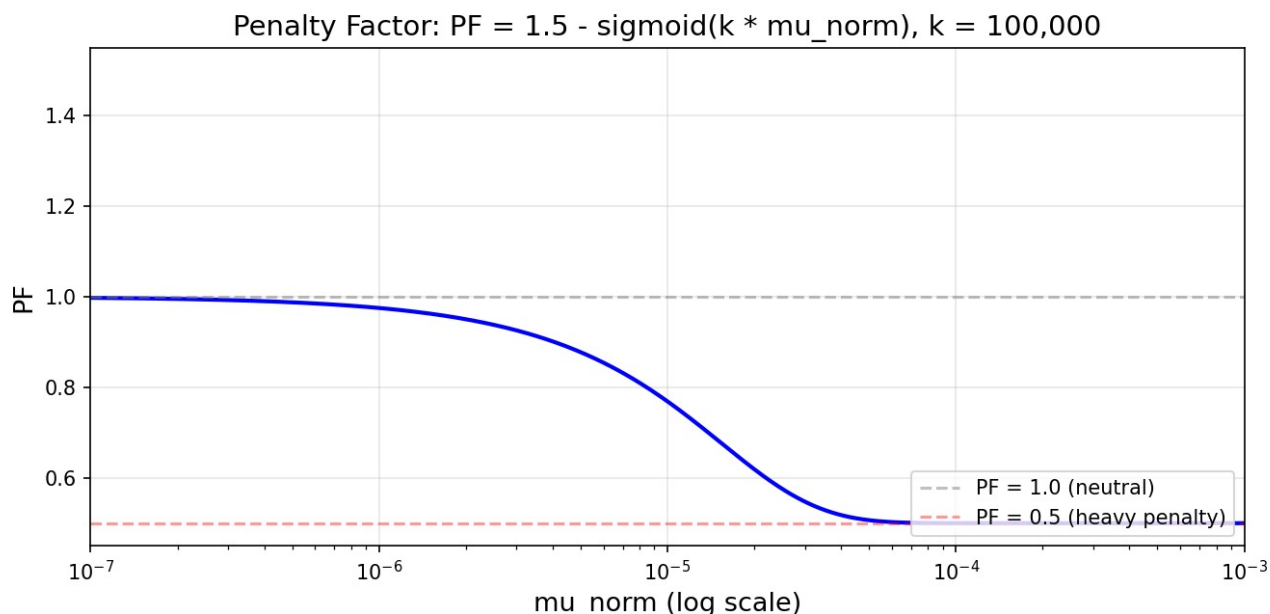
- **Modely:** Musíte použít `torchvision.models.resnet18(pretrained=True)` a `timm.create_model('vit_tiny_patch16_224', pretrained=True)`. Žádné jiné předtrénované modely nejsou povoleny.
- **Transformační pipeline (vynucená při vyhodnocení):** `Resize(256) → CenterCrop(224) → Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])`. See Section 3 of `baseline.ipynb` pro podrobnosti.
- **Rozlišení perturbace:** Musí odpovídat **původnímu** rozlišení nezpracovaného obrázku (nikoli 224×224). Tensor se přičte k nezpracovanému obrázku *před* transformační pipeline.
- **Výstupní formát:** Pouze soubory `.pt` — žádné PNG/JPG. Tenzory se přičtou k nezpracovanému obrázku a hodnoty pixelů se před předzpracováním oříznou na `[0, 1]`.
- **Názvy souborů:** Ploché seznam, striktní formát `{index}_a.pt / {index}_b.pt`. Uvnitř souboru zip nesmějí být žádné podadresáře.
- **Knihovny:** `torch`, `torchvision`, `timm`.

5. Bodování

Konečné skóre se vypočítá následovně. Necht' M je počet obrázků v dané části, $Score_A$ počet úspěšných perturbací typu A a $Score_B$ počet úspěšných perturbací typu B:

$$S_{final} = \frac{Score_A + Score_B}{2M} \times PF$$

PF je funkce navržená tak, aby penalizovala perturbace s vysokou normou a byla velmi citlivá v blízkosti horní meze výkonu. Je omezena na rozsah 0.5 až 1. Úplnou implementaci lze nalézt v oddílu 8 souboru `solution.ipynb`.



Obrázek: Křivka penalizační funkce.

6. Kontrola odevzdání

Notebook obsahuje kontroly, které vás upozorní na problémy s formátováním; nacházejí se v oddílu 7 notebooku `solution.ipynb`.

7. Lokální testování

`solution.ipynb` obsahuje úplný funkční příklad. Načte veřejná data, oba modely i oficiální bodovací nástroj a запиše soubor ZIP k odevzdání. Než začnete, přečtěte si jej.

8. Jak odevzdat řešení

- Uložte své změny do `solution.ipynb`.
- Otevřete kartu Git v levém postranním panelu JupyterLab.
- Přidejte `solution.ipynb` do **stage** (ikona + vedle něj).
- Zadejte zprávu commitu a klikněte na **Commit**.
- Kliknutím na ikonu cloudu se šipkou nahoru provedte push.
- Vraťte se na tuto stránku soutěže a klikněte na **Submit**.

Odevzdejte právě jeden soubor s názvem `solution.ipynb`.